ASIAN UNIVERSITY FOR WOMEN

ICP Final Project Report

Project Title : Battle Quest: A Console-Based Turn-Based Combat Game

Student 1 Details
Name: Suha Juhaira Zaman
ID: 252014
AUW Email: auw252014@auw.edu.bd

Student 2 Details
Name: Ipsita Sarwar Tanha
ID: 252041
AUW Email:auw252041@auw.edu.bd

Submission Date: 13th August, 2026

1. **Abstract**

Battle Quest is a console-based turn-based combat game developed using the C programming language. The game allows a player to enter their name and battle against a computer-controlled Goblin. The player can choose between three actions: attacking, healing, or defending. Random values are used to determine damage and healing, making each turn different. ASCII art is also included to make the game more visually engaging. The project was developed to apply fundamental C programming concepts such as variables, arrays, conditional statements, loops, user input, arithmetic operations, and random number generation.

2. **Introduction**

Computer games provide an engaging way to apply programming concepts because they require user interaction, decision-making, and logical conditions. For this project, a simple combat game called *Battle Quest* was developed using the C programming language. The main objective of the project was to create an interactive game while applying the programming concepts learned during the course. The game begins by asking the player to enter their name. The player then faces a Goblin with 100 HP. During each turn, the player can choose to attack, heal, or defend. The battle continues until either the player's HP or the enemy's HP reaches zero. Another objective was to make the program more interesting than a basic text-based program. ASCII characters and attack animations were therefore included to provide a more game-like experience without requiring advanced graphical libraries.

3. **Implementation**

The project was implemented using the C programming language. The program uses the standard input/output library and the standard library for generating random values.

At the beginning of the program, variables are initialized for the player's HP, enemy HP, player name, and the player's selected action. Both the player and the Goblin begin with 100 HP.

The player is asked to enter their name using scanf(). A while loop is then used to keep the battle running as long as both the player and enemy have HP greater than zero.

During each turn, the player is presented with three options:

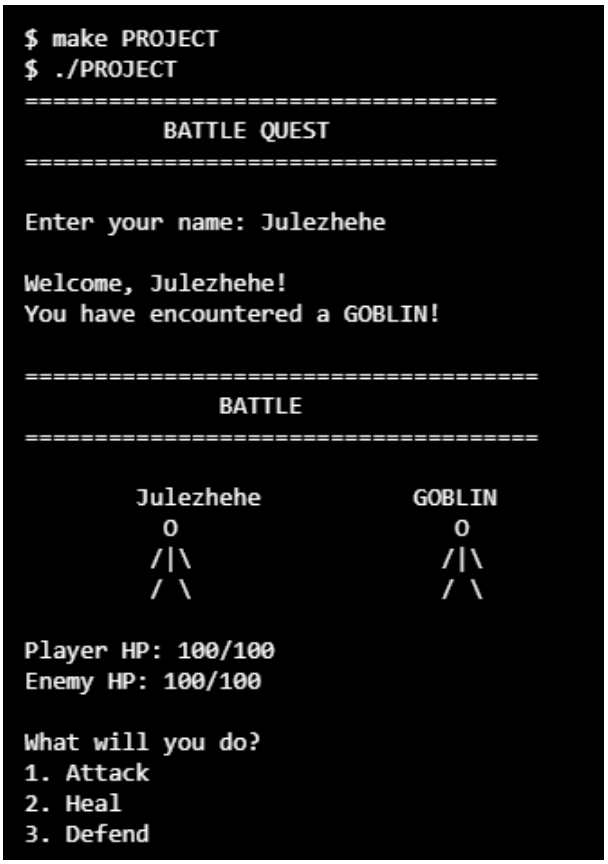
1. **Attack** – The player deals a random amount of damage between 10 and 20.
2. **Heal** – The player restores a random amount of HP between 10 and 20.
3. **Defend** – The player receives a reduced amount of damage from the enemy.

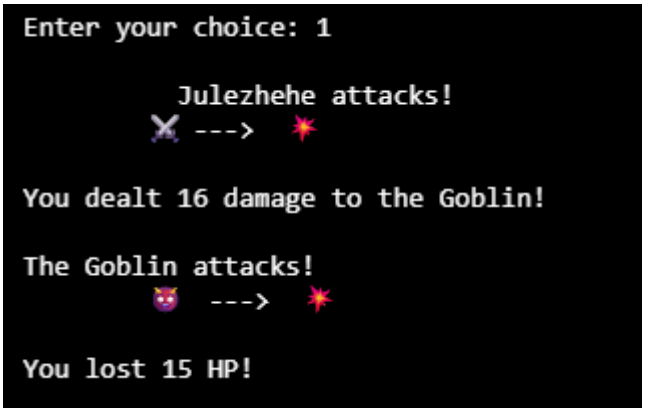
The rand() function is used to generate different damage and healing values. Conditional statements such as if, else if, and else are used to determine which action the player selected.

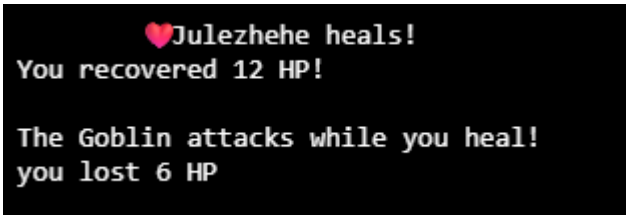
The program also checks that the player's HP does not exceed 100 after healing. Once the battle ends, another conditional statement determines whether the player has won or lost.

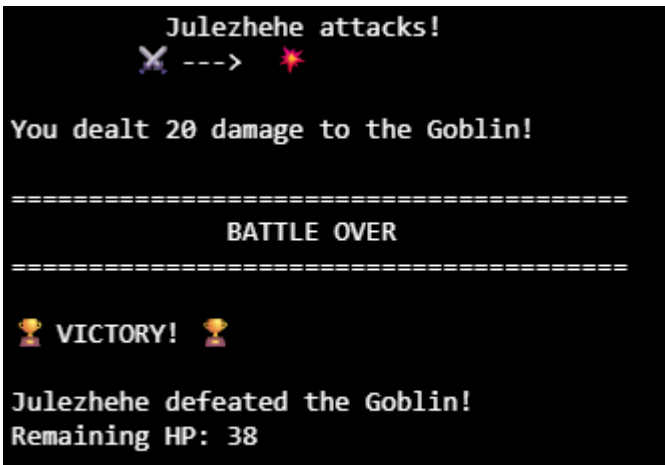
ASCII art was incorporated using printf() statements. This provides a visual representation of the player, enemy, and attacks while keeping the implementation within the basic C concepts covered in the course.

4. **Output**









5. **Challenges and Limitations**

   One of the main challenges was designing the game logic so that the player's actions correctly affected both the player's and enemy's HP. Different conditions had to be considered, such as preventing the player's HP from exceeding 100 and ensuring that the enemy does not attack after being defeated.

   Another challenge was using random numbers for damage and healing while keeping the values within reasonable limits. The rand() function was used to create variation between turns.

   The project also has some limitations. The current version contains only one enemy, the Goblin, and does not include different levels or character abilities. The game is console-based, so it does not contain graphical images or advanced animations. Additionally, the random number generation could be improved in future versions by using a random seed.

6. **Conclusion and Discussion**

   Battle Quest successfully achieved its main objective of creating an interactive combat game using fundamental C programming concepts. The project provided practical experience with variables, user input, loops, conditional statements, arithmetic operations, strings, and random number generation.

   The inclusion of different player actions and ASCII art made the program more interactive and visually engaging than a basic console application. Working on the project also helped demonstrate how individual C programming concepts can be combined to create a complete functioning program.

   Overall, Battle Quest was a useful application of the concepts learned during the course and provided an opportunity to develop problem-solving and programming skills.

7. **Future Work**

   Several improvements could be made to BattleQuest in the future. The game could include multiple enemies, with each enemy having different HP and attack strengths. Different player abilities could also be added to provide more strategic choices.

   Other possible improvements include:

- Adding different character classes
- Adding multiple battle rounds
- Introducing a final boss
- Adding a scoring system
- Creating more detailed ASCII animations
- Adding critical attacks with special effects

- Allowing the player to choose different weapons
- Improving random number generation using srand()
- Adding a restart option after the game ends

8. References

Since the BattleQuest program was developed by the project members using concepts learned in the C programming course, no external code was directly copied.

- Course lecture materials and notes on C programming.
- C programming concepts practiced during laboratory sessions.

9. Individual Contributions

Ipsita Sarwar Tanha

- Contributed to the initial concept and design of the Battle Quest game.
- Helped design the player actions, including attack, heal, and defend.
- Worked on the implementation of player input and HP management.
- Assisted with the use of conditional statements and loops.
- Helped test the program and identify errors.
- Assisted in preparing and reviewing the final project report.

Suha Juhaira Zaman:

- Contributed to the development of the battle logic and enemy behavior.
- Assisted with implementing random damage and healing using rand().
- Worked on the game's win and defeat conditions.
- Helped test different gameplay situations and correct errors.
- Contributed to the visual formatting and ASCII art used in the console.
- Assisted in reviewing and improving the final code.
- Contributed to preparing and reviewing the final project report.